



From intake to review.

Six steps today. A clear path to production.

Evidence dated September 23, 2026 · Works offline

READY NOW · DEVELOPMENT

Intake → review → history

The four-screen app is saved and **unpublished**. Synthetic workflow checks passed.

NOT YET

Production delivery

There is no Send or Release action.
APPROVED means reviewed, not sent or shipped.

NEXT · PROPOSED

Azure SQL → PostgreSQL

Phase 2 assumes a published app.
PostgreSQL production is targeted for **end of Q4 2026**, after acceptance.

Evidence date: September 23, 2026. This guide describes the recorded development app and proposed production plan. It is not a live service-status page. Sharing this file does not grant access to the app.

Use the app

For an authorized tester: have the owner confirm the development laptop, API, database and gateway are running. Open **MILSTRIP Intake Dev** in Power Apps Studio, select **Preview**, and use approved test input.

1 Paste the source

On **MILSTRIP intake**, paste the original email or ticket text. Keep the source unchanged; obtain missing business values rather than guessing.

2 Submit once

Select **Submit intake** and wait for results. Keep the **Request ID**: it is your reference for this intake. If no result arrives, use the recovery steps below before submitting again.

3 Check every record

Select **Load / refresh results**, then **Inspect / review** beside a record. Read its status and issues; use **Next results page** when enabled. Accepted canonical output should be 80 characters; the field is read-only.

4 Save a decision

Choose **APPROVED** or **REJECTED**, enter a reason of 1–1,000 characters, and select **Save review**. Validation-REJECTED records cannot be approved. Resolve warnings from business evidence; reject or escalate unresolved issues.

Approval records your review. It does not release an order.

5 Confirm it saved

Look for the saved message and updated review version. Return through **Results** and refresh before relying on displayed review status.

6 Check the history

Select **History** → **Load / refresh history**; use **More history** when enabled. Confirm the decision and keep the Request ID. The current audit actor is the shared development connection, not your individual sign-in.

You are finished when the saved decision appears in history. The existing authorized production process remains separate.

If something goes wrong

▼ Submission timed out or the outcome is unknown

Keep the original text and the **source ID** shown in the message. Select **Load recent requests**, then **More requests** if needed, and match that source ID. Open the matching request if found. If absent, ask the owner to confirm it was not created before selecting **Allow new submission**. A timeout does not mean nothing was saved.

▼ A record is REJECTED or REQUIRES_REVIEW

Read the issue and obtain authoritative corrections. For invalid input, submit corrected source as a **new intake** and keep both Request IDs. Do not trim or rebuild the canonical string. For a warning, review the evidence; reject or escalate if it remains unresolved.

▼ A review did not confirm, or another review changed the record

For an unknown review outcome, stay on the record and select **Retry same command** when enabled. Keep the retained decision and reason unchanged. For a version conflict, select **Reload after conflict**, inspect the current review and retained reason, then decide again if appropriate.

▼ Results will not load, or the app was closed

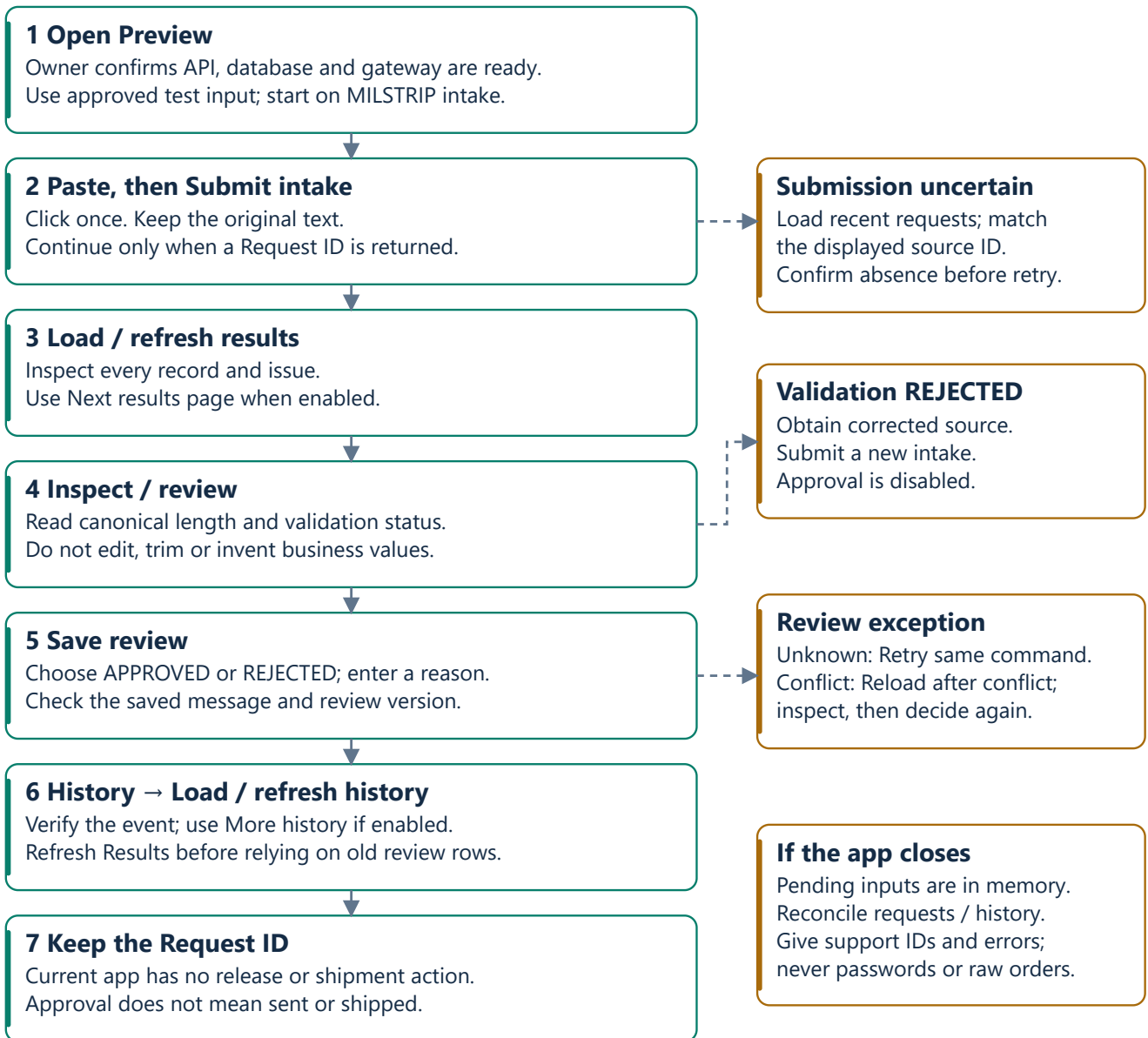
After service recovery, retry the relevant load/refresh action. Treat retained rows as potentially stale. If the app restarted, reconcile recent requests and history: unsaved input and pending commands are held in memory. Give support the Request ID, record number, approximate time and visible error; exclude passwords and unredacted order screenshots.

See the operator flowchart

End-user SOP: use the current app

CURRENT DEV

Authorized development testing only. Exact button names match the existing screens.



MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

The same operator journey, with recovery branches.

Understand the workflow

Three different questions keep the workflow clear:

Validation

Does the record pass format and business-rule checks? A request can

Review

What did the reviewer decide, and why? The original evidence stays

Delivery

Was an order handed off and acknowledged? This capability is

contain records with different results.

intact.

proposed for Phase 2.

Today, the app uses the existing solution, custom connector and gateway to reach a laptop API and local PostgreSQL. The parser preserves accepted positions and pads eligible records to 80 characters. It does not invent business data or check production stock/address references.

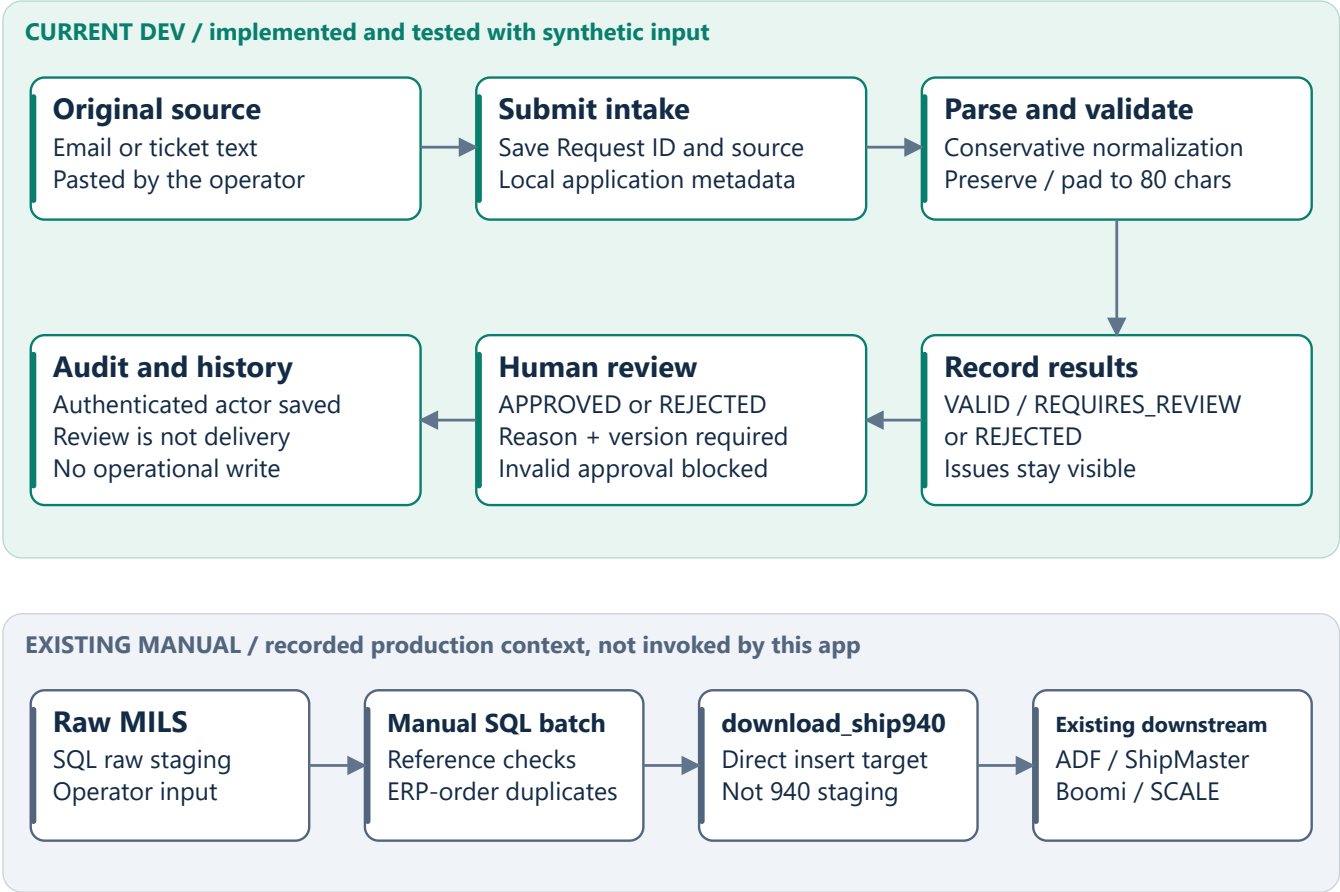
The separate manual process loads raw MILS staging, checks references and duplicate orders, and writes directly to the existing order table. The downstream flow remains unchanged; it was not exercised by the app's synthetic checks.

See the current functional flow

Functional flow: what the app does today

CURRENT DEV

The app ends at review and audit. The manual production route is separate.



MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

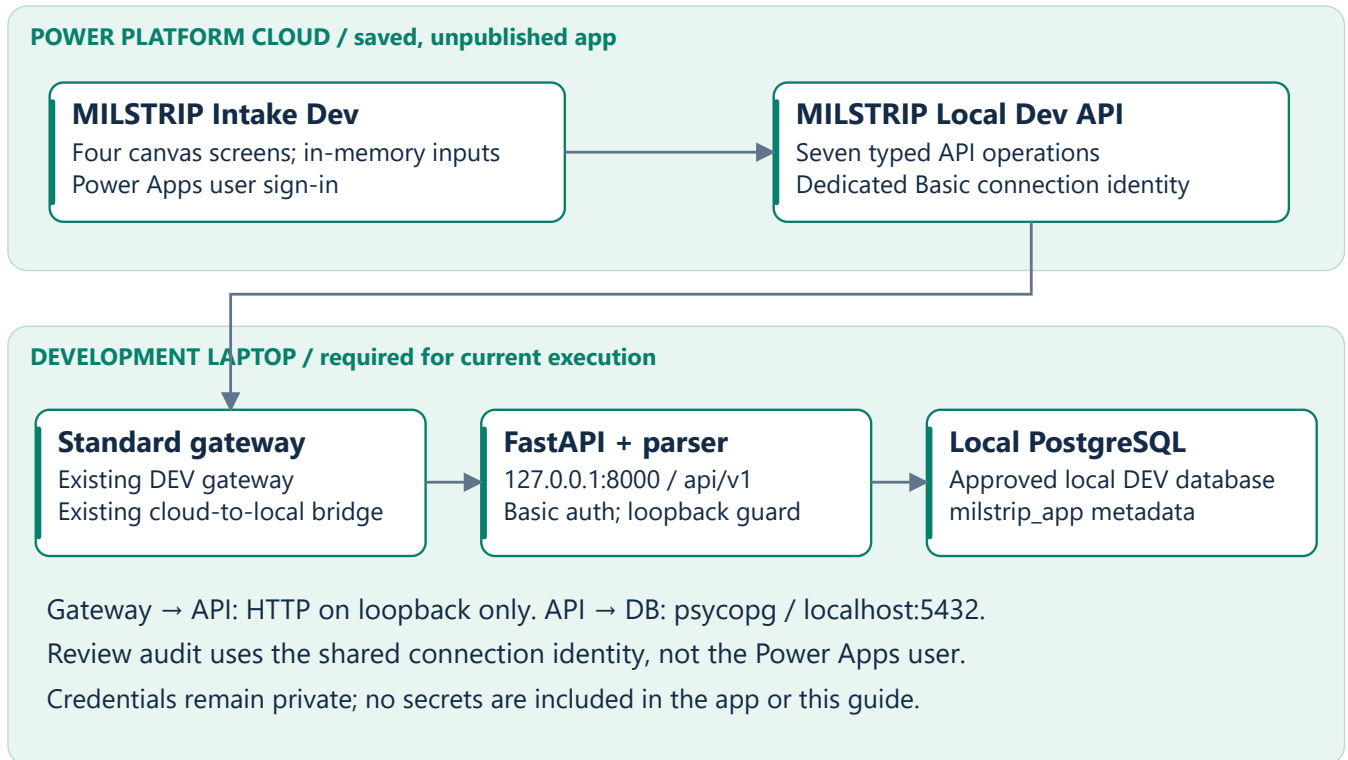
Current development workflow and separate manual production path.

See the current technical architecture

Technical architecture: current development

CURRENT DEV

Existing Default environment, MILSTRIP solution, app, connector and gateway.



NO RUNTIME LINK TO PRODUCTION SQL / no legacy writer, CSV export or FTP delivery

MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

Current cloud connector, development laptop API and local PostgreSQL.

Seven authenticated API operations support intake, request lookup, results, reviews and history. Basic authentication identifies the development connection. The API enforces the approved local database boundary. Publishing the app alone will not host the API, add per-user audit identity or enable Azure SQL.

Still to verify: accessibility/layout, canvas conflict and outage recovery, multi-page navigation, and shared-user authorization. The underlying API retry/conflict cases have standalone test evidence; that does not complete UI acceptance.

Phase 2: two production periods

Planning assumption: the app is published. Keep the same operator journey and existing app lineage while qualifying production hosting, identity and one delivery route.

1 · Use existing Azure SQL

Move the API off the development laptop; add individual-user authorization, a SQL Server persistence adapter, live reference checks and controlled release. Begin with a bounded pilot using existing SQL capacity.

Ready when: API/UI failure paths, duplicate prevention, restore drills and operator acceptance pass. This requires implementation, not a connection-string swap.

2 · Move to accepted PostgreSQL

Retain the app/API contract. Migrate records, review history, audit identities and pending release commands; qualify database behavior and downstream acknowledgements. Switch production ownership once.

Target: December 31, 2026, subject to migration acceptance. The proposed Stage and Production services share one VM; that is isolation, not high availability.

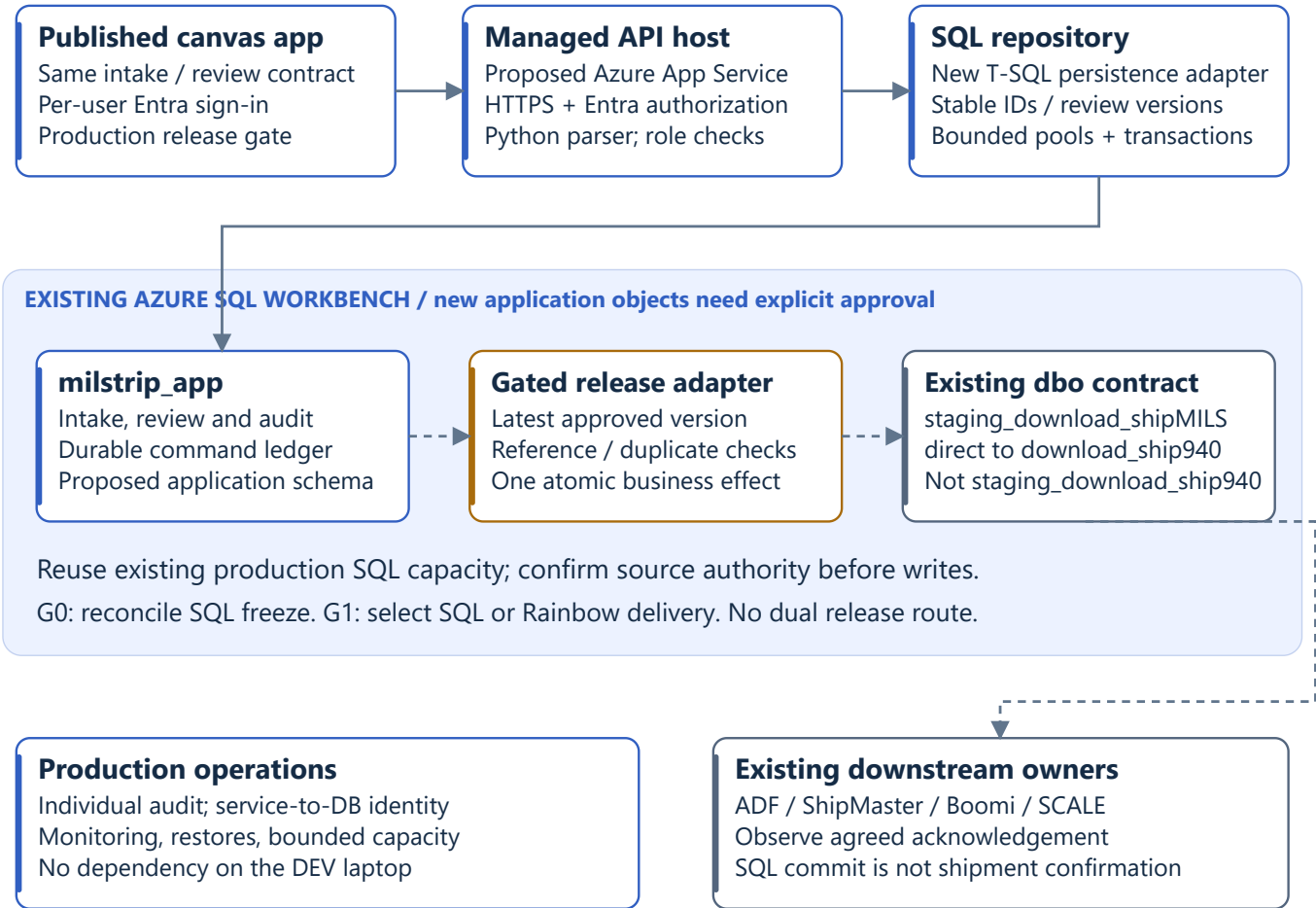
Three decisions come first

1. **Confirm SQL authority.** Migration records describe a freeze while the operating plan still uses SQL. Reconcile this before writes; new SQL activity requires an updated accepted migration baseline.
2. **Choose one delivery route.** Direct SQL handoff is the estimating baseline; the earlier Rainbow CSV/FTP proposal remains unresolved. Do not release the same order through both.
3. **Approve production ownership.** Set identities, hosting, workload limits, backup/recovery targets and the people responsible for acceptance.

P2.1 architecture: Azure SQL production

PROPOSED

PROPOSED. Published app is assumed; production hosting and adapters are not implemented.



MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

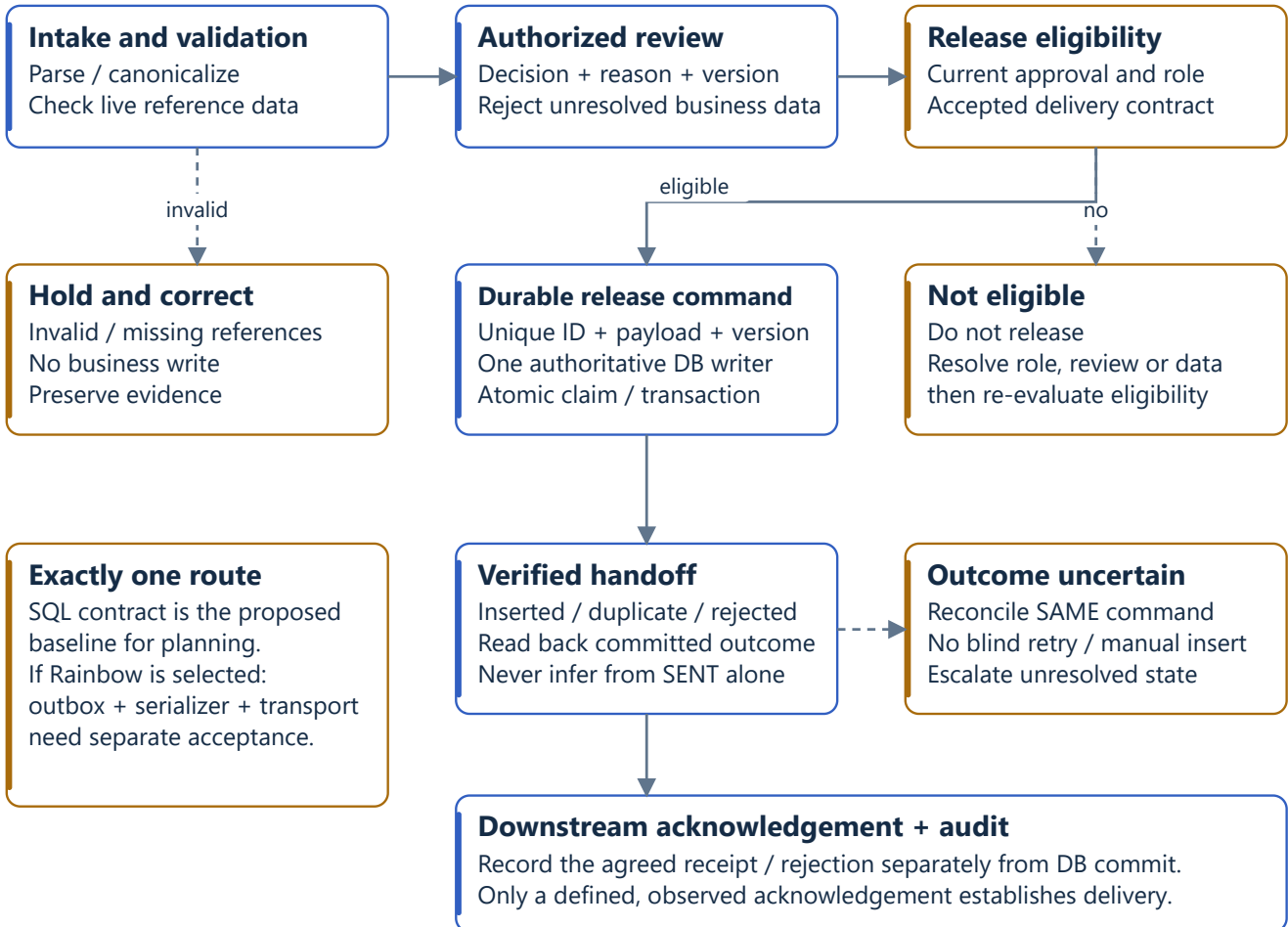
Proposed Azure SQL production architecture and controlled release boundary.

See the future review → release → acknowledgement flow

Phase 2 flow: review, release, acknowledge

PROPOSED

PROPOSED controls and states. None of the release/delivery actions exists today.



MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

Proposed release flow; these controls do not exist in the current app.

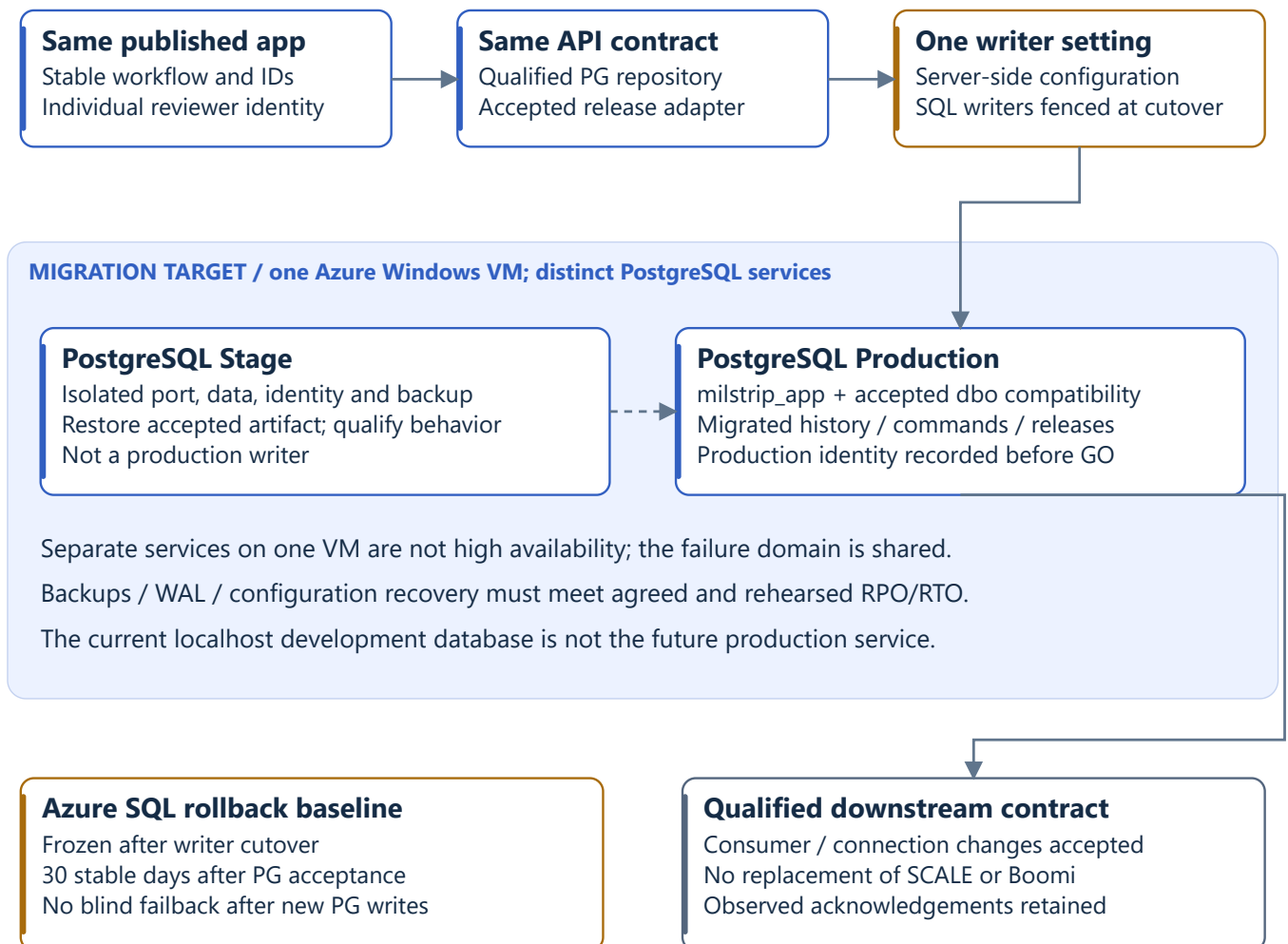
A future release action needs a current approval, live reference checks, the right role and a durable command ID. Reconcile an uncertain outcome using that same command. A database commit or legacy SENT flag does not prove shipment; observe the agreed downstream acknowledgement separately.

See the PostgreSQL architecture

P2.2 architecture: PostgreSQL production

PROPOSED

PROPOSED. Switch only after migration, business parity and production acceptance.



MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

Proposed PostgreSQL production after migration and operator acceptance.

Two clocks, one handover

Before cutover: two months of controlled qualification while SQL remains authoritative. Start by October 31 for a full window ending December 31; late readiness moves the date unless the owner revises the gate.

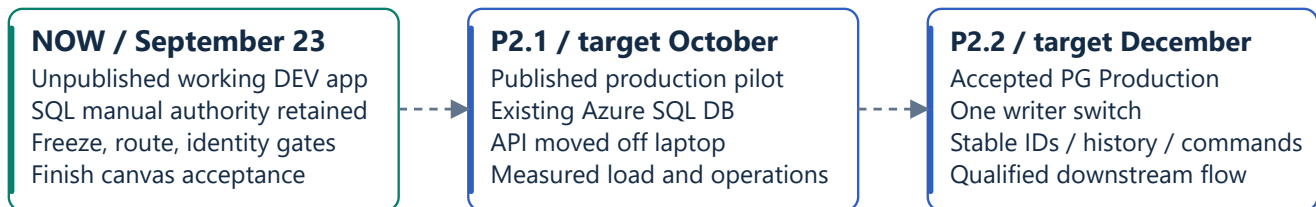
After PostgreSQL acceptance: retain frozen SQL for 30 stable days before a separate retirement decision. A December 31 launch carries retention into January 2027. End-Q4 retirement would need acceptance around December 1.

See the transition timeline

Two periods: qualify, switch, stabilize

PROPOSED

Q4 2026 is the owner's target. Dates never override the acceptance gates.



BEFORE CUTOVER / two-month MILSTRIP controlled comparison

SQL remains authoritative; PG comparison does not create duplicate production effects.

Start no later than October 31 for a full two-month window ending December 31.

Late readiness means a later date or an explicit gate change, not skipped evidence.

AFTER PG PRODUCTION ACCEPTANCE / 30 stable days of SQL retention

Frozen SQL is a rollback baseline, not an automatic live fallback for new PG writes.

December 31 go-live pushes this window into January 2027.

End-Q4 retirement instead requires acceptance around December 1 and a stable window.

Cutover sequence

Quiesce → capture → reconcile → switch once → observe → separately approve retirement

MILSTRIP | September 23, 2026 | Public guide v1.1.0 | Recorded state, not live service status

Qualification, one writer switch and stabilization are separate steps.

▼ Capacity, cutover and recovery: the essentials

Measure before scaling. Record peak users, batch size, API latency, lock waits, database CPU/IO, backlog and growth. For Azure SQL, test twice the measured expected peak; tune batches, queries and pools before buying capacity. Re-run the suite on PostgreSQL and address its indexes, pools and maintenance from evidence. These are proposed checks, not current service guarantees.

Cut over in order: baseline → rehearse → stop all writers → capture → reconcile → switch once → observe. Include new app/audit/release objects in the migration scope. Reconcile pending commands, qualify downstream connections, fence old SQL writers, and retain independently tested backups.

Recover without losing new work. Before PostgreSQL production writes, a validated return to the frozen SQL baseline is possible. After new PG writes, pause and reconcile those changes or repair PG. A blind connection switch to stale SQL risks loss and duplicate orders.

Next work: finish canvas acceptance and resolve the three decisions, then implement the Azure SQL adapter against a non-production target. Production deployment remains a separate accepted release.

About this edition

Version **1.1.0** · Public reading edition · Evidence dated **September 23, 2026**.

Current claims come from recorded canvas execution, source controls and API tests. The manual route comes from the September 21 legacy review; migration assumptions come from the September 3 migration master and accepted roadmap. This edition does not re-inspect the tenant or production database. The implementation team retains detailed evidence and engineering decisions separately.

All seven charts are embedded. Expand a chart to read it, use **View larger** to zoom, or download its SVG. The HTML works offline; print includes expanded details. This documentation release does not publish or install the Power App.

Public documentation edition · September 23, 2026 · Current app: unpublished development draft. Phase 2: proposed. Sharing this guide does not share or publish the app.